ZERO HOUR

Product Requirements Document

AI-Powered Last-Minute Productivity Companion

Version	1.0.0
Status	In Development
Hackathon	Vibe2Ship — BlockseBlock
Problem	The Last-Minute Life Saver
Developer	Priyanshu Ghosh
Stack	Spring Boot · React · Gemini 2.5 Flash · Firebase · Cloud Run
Deadline	June 29, 2025

ZeroHour is a multi-agent AI productivity companion that helps students, professionals, and entrepreneurs avoid missed deadlines through proactive planning, intelligent scheduling, and context-aware nudges — powered by Gemini 2.5 Flash and the Google ecosystem.

1 Executive Summary

Students, professionals, and entrepreneurs consistently miss deadlines, assignments, meetings, bill payments, and important commitments. Existing productivity tools rely on passive reminders that are trivially ignored and do nothing to help users actually complete tasks.

ZeroHour moves beyond reminders. It is an AI agent system that actively plans, prioritizes, schedules, and nudges — adapting in real time to the user's situation, especially in high-stress last-minute scenarios. The flagship feature, Panic Mode, lets users describe a crisis in natural language and receive a full survival plan pushed directly to their Google Calendar.

Core value proposition

From passive	→	**Active AI planning**
From ignored reminders	→	**Proactive agent nudges**
From vague goals	→	**Concrete subtask breakdowns**
From manual scheduling	→	**Auto-pushed to Google Calendar**
From rigid apps	→	**Conversational, adjustable flow**

2 Goals & Non-Goals

Goals

- ✓ Deliver a working multi-agent AI system with 4 distinct agents demonstrating real agentic depth
- ✓ Implement Panic Mode as a conversational, adjustable, confirmation-gated flow
- ✓ Integrate Google Calendar API for real event creation from AI-generated plans
- ✓ Deliver Gmail nudge emails + in-app notifications for proactive deadline management
- ✓ Deploy publicly on Google Cloud Run using Gemini 2.5 Flash (free tier)
- ✓ Score high on all 7 evaluation criteria — especially Agentic Depth (20%) and Google Tech (15%)

Non-goals (out of scope for v1)

- ✗ Mobile app (web-first, responsive)
- ✗ Offline mode
- ✗ Team / collaborative task management
- ✗ Paid API usage (everything must run on free tiers)
- ✗ Custom ML model training
- ✗ Integration with third-party tools (Notion, Jira, etc.)

3 User Personas

P1 · The Panicking Student

College student with an exam in 3 hours who hasn't studied. Needs immediate structure, not motivation.

Key needs: Panic Mode · Subtask breakdown · Countdown timer

P2 · The Overwhelmed Professional

Working professional with multiple deadlines converging. Needs intelligent prioritization.

Key needs: Task prioritization · Calendar integration · Email nudges

P3 · The Deadline-Missing Entrepreneur

Founder who misses bill payments and pitch deadlines due to context-switching.

Key needs: Proactive nudge agent · Goal → task explosion · Calendar blocking

4 Agent Architecture

ZeroHour is built on a 4-agent pipeline orchestrated by a central Spring Boot service. Agents run sequentially — each agent's output is the next agent's input. The UI streams agent status in real time via Server-Sent Events.

Planner Agent

`Class: PlannerAgent.java`

Receives raw user goal + deadline. Calls Gemini 2.5 Flash with a structured prompt. Outputs a JSON list of time-boxed subtasks with estimated durations and rationale.

Triggers on: task creation, Panic Mode activation

Prioritizer Agent

`Class: PrioritizerAgent.java`

Receives subtask list from Planner. Calls Gemini to score urgency × impact. Outputs ranked subtasks with CRITICAL / HIGH / MEDIUM / LOW labels.

Triggers on: after PlannerAgent completion

Scheduler Agent

`Class: SchedulerAgent.java`

Receives confirmed subtask list. Calls Google Calendar API v3 to create events. Packs subtasks consecutively from current time. Stores event IDs in Firestore.

Triggers on: user clicks confirm button (never auto-fires)

Nudge Agent

`Class: NudgeAgent.java`

Spring @Scheduled cron (every 15 min). Scans Firestore for approaching deadlines. Sends in-app notifications + Gmail API emails at 24h, 6h, and 1h thresholds.

Triggers on: cron schedule, checks nudges.sent to prevent duplicates

5 Feature Specifications

F01 Google OAuth 2.0 Authentication

CRITICAL

Users sign in with their Google account. OAuth scopes include Calendar and Gmail access, so no additional permission prompts are needed later. Tokens are stored encrypted in Firestore and refreshed automatically before every API call.

Acceptance criteria

- ✓ User can click 'Sign in with Google' and complete OAuth flow
- ✓ Access token and refresh token stored in Firestore users collection
- ✓ Token auto-refreshed when expired, transparent to user
- ✓ Logout clears session and tokens
- ✓ Unauthenticated routes redirect to landing page

Tech notes

- • spring-boot-starter-oauth2-client handles the OAuth flow
- • Scopes: openid, email, profile, calendar, calendar.events, gmail.send
- • Store tokens in Firestore (not HTTP session) for Cloud Run stateless compatibility

F02 ■ Panic Mode — Conversational Crisis Flow

CRITICAL

The hero feature. User describes a crisis in natural language. ZeroHour asks 2-3 targeted follow-up questions to gather context. User can add information or change any answer freely at any point. Gemini generates a survival plan. User sees an editable plan preview before confirming. On confirmation, Scheduler Agent pushes to Google Calendar.

Acceptance criteria

- ✓ User can trigger Panic Mode from any screen via red emergency button
- ✓ System asks follow-up questions (max 3 turns before plan generation)
- ✓ User can type 'add more info' or correct any previous answer at any time
- ✓ Plan is shown as editable cards — user can rename subtasks, change durations, reorder, delete, add
- ✓ Confirmation button only appears after user reviews plan
- ✓ Calendar events created only after explicit confirmation click
- ✓ Countdown timer shown on completion screen
- ✓ Panic session saved to Firestore for history

Tech notes

- • Multi-turn conversation stored in panic_sessions.conversationJson (serialized array)
- • Gemini prompt returns { ready: false, question: string } or { ready: true, summary: string }
- • Frontend uses SSE to show agent thinking panel during plan generation
- • Plan preview uses drag-to-reorder (react-beautiful-dnd or similar)

Agent: PlannerAgent → PrioritizerAgent → (user confirms) → SchedulerAgent

F03 Task Creation with AI Planning

CRITICAL

User creates a task with a title, description, and deadline. PlannerAgent automatically breaks it into subtasks using Gemini. PrioritizerAgent ranks them. User sees the AI-generated plan immediately. This flow also feeds into the Panic Mode experience.

Acceptance criteria

- ✓ Task form: title (required), description (optional), deadline (required, datetime picker)
- ✓ On save, PlannerAgent called automatically — subtasks appear within 3 seconds
- ✓ PrioritizerAgent ranks subtasks, priority badges shown on each card
- ✓ Agent thinking panel visible during generation (SSE stream)
- ✓ User can edit generated subtasks before confirming to Calendar
- ✓ Task saved to Firestore tasks collection with status PENDING

Tech notes

- • POST /api/tasks triggers orchestrator synchronously (or async with SSE progress)
- • Gemini response parsed as JSON — fallback shown if malformed
- • Subtasks written to Firestore subtasks collection with taskId FK

Agent: PlannerAgent → PrioritizerAgent

F04 Google Calendar Integration

CRITICAL

After user confirms a plan (from task creation or Panic Mode), SchedulerAgent creates real Google Calendar events for each subtask. Events are color-coded red for urgency. Calendar event IDs are stored in Firestore. User gets a link to view events in Google Calendar.

Acceptance criteria

- ✓ Each confirmed subtask becomes one Google Calendar event
- ✓ Events packed consecutively starting from current time
- ✓ Event title format: [ZeroHour] {subtask title}
- ✓ Event description includes parent task name and ZeroHour attribution
- ✓ Event color set to red (colorId: 11) for urgency visibility
- ✓ Google Calendar link shown to user after creation
- ✓ Events deletable from ZeroHour (also deletes from Google Calendar)
- ✓ Timezone: Asia/Kolkata (IST) for all events

Tech notes

- • Google Calendar API v3, google-api-services-calendar dependency
- • Uses user's stored OAuth access token + refresh logic
- • calendar_events Firestore collection stores googleEventId + taskId mapping

Agent: SchedulerAgent

F05 Task Dashboard

HIGH

Main screen after login. Shows all user tasks with priority badges, deadline countdowns, and status indicators. Supports filtering by priority and status. Quick-add task button and Panic Mode trigger button always visible.

Acceptance criteria

- ✓ Task list with cards showing: title, priority badge, deadline, status, subtask count
- ✓ Color-coded priority: red=CRITICAL, amber=HIGH, blue=MEDIUM, gray=LOW
- ✓ Deadline countdown (e.g. '2h 34m left') shown in monospace font
- ✓ Filter tabs: All / Today / Overdue / Done
- ✓ Sort by: deadline (default), priority, created date
- ✓ Empty state with helpful CTA when no tasks
- ✓ Panic Mode red button always accessible in header/FAB
- ✓ Notification bell with unread badge in header

Tech notes

- • GET /api/tasks returns all tasks for authenticated user
- • Frontend sorts/filters client-side (no extra API calls)
- • Overdue tasks highlighted with red border

F06 Task Detail View

HIGH

Detailed view of a single task. Shows all subtasks with their priorities, durations, and completion status. Links to Google Calendar events. Shows nudge schedule. User can mark subtasks as done and update task status.

Acceptance criteria

- ✓ Subtask list with checkbox to mark done
- ✓ Priority badge and duration shown per subtask
- ✓ Linked Google Calendar events shown with external link
- ✓ Nudge schedule shown: which nudges sent, which pending
- ✓ Task status update: PENDING → IN_PROGRESS → DONE
- ✓ Delete task (also deletes calendar events and cancels nudges)
- ✓ Edit task title and deadline
- ✓ Progress bar showing subtask completion percentage

Tech notes

- • GET /api/tasks/{id} returns task + subtasks + calendar events + nudge schedule
- • PUT /api/tasks/{id} for status updates
- • Calendar event deletion via Google Calendar API + Firestore cleanup

F07 Intelligent Task Prioritization

HIGH

PrioritizerAgent uses Gemini to score each subtask on urgency × impact matrix. Priority labels are CRITICAL, HIGH, MEDIUM, LOW. The agent considers deadline proximity, estimated duration, and task dependencies when ranking.

Acceptance criteria

- ✓ Every task gets AI-assigned priority labels on all subtasks
- ✓ Priority shown as colored badge on every task and subtask card
- ✓ CRITICAL tasks shown at top of dashboard regardless of sort
- ✓ Priority recalculated if user changes deadline
- ✓ Agent logs visible in thinking panel during prioritization

Tech notes

- • PrioritizerAgent prompt returns JSON with priority + priorityReason per subtask
- • priorityReason shown as tooltip on hover over priority badge
- • Re-prioritization: PUT /api/tasks/{id}/reprioritize endpoint

Agent: PrioritizerAgent

F08 Proactive Nudge System

HIGH

NudgeAgent runs every 15 minutes as a background cron job. It scans all tasks approaching their deadlines and sends proactive nudges via two channels: in-app notifications and Gmail email. Each nudge is sent only once per threshold (24h, 6h, 1h before deadline).

Acceptance criteria

- ✓ In-app notification appears in bell dropdown with task title + deadline
- ✓ Gmail email sent with task name, deadline, and link back to ZeroHour
- ✓ Three nudge thresholds: 24h before, 6h before, 1h before deadline
- ✓ Nudge not re-sent if already sent (nudges.sent = true check)
- ✓ User can dismiss in-app notifications (mark as read)
- ✓ Email includes 'Open in ZeroHour' deep link
- ✓ Nudge history visible in task detail view

Tech notes

- • Spring @Scheduled(fixedRate = 900000) — every 15 minutes
- • Gmail API used for email sending (no SMTP, no third-party service)
- • Firestore nudges collection tracks sent status to prevent duplicates
- • NudgeAgent runs asynchronously via @Async to not block request threads

Agent: NudgeAgent

F09 Live Agent Thinking Panel

HIGH

While agents work, the UI shows a live log panel displaying which agent is active, what it is doing, and when it completes. Each agent appears as a row with animated spinner → green checkmark transition. Implemented via Server-Sent Events (SSE).

Acceptance criteria

- ✓ Agent log panel appears automatically when any agent is triggered
- ✓ Each agent shown as row: [spinner/check] [Agent Name] [status badge] [message]
- ✓ Real-time updates via SSE — no polling
- ✓ Panel collapses automatically when all agents complete
- ✓ Summary card shown after collapse (e.g. '5 subtasks created, 3 calendar events pushed')
- ✓ Panel accessible again by clicking 'View last run' in task card

Tech notes

- • GET /api/agents/stream/{sessionId} — SSE endpoint, Content-Type: text/event-stream
- • AgentOrchestrator emits events to SseEmitter after each agent step
- • Frontend reconnects automatically if SSE connection drops
- • Agent log stored in Firestore for replay

F10 In-App Notification Center

MEDIUM

Notification bell in the header shows unread count badge. Clicking opens a dropdown with recent nudges, agent completion alerts, and calendar sync confirmations. All notifications stored in Firestore and persist across sessions.

Acceptance criteria

- ✓ Bell icon with unread count badge (red dot if >0)
- ✓ Dropdown shows last 20 notifications
- ✓ Notification types: nudge, agent complete, calendar sync, overdue alert
- ✓ Mark individual notifications as read
- ✓ Mark all as read button
- ✓ Clicking notification navigates to relevant task
- ✓ Notifications persist across sessions (Firestore)

Tech notes

- • GET /api/notifications — list user notifications
- • PUT /api/notifications/{id}/read — mark read
- • Firestore real-time listener on frontend for instant updates (onSnapshot)

F11 Landing Page & Onboarding

MEDIUM

Public landing page explaining ZeroHour's value proposition. Clean dark design with Panic Mode demonstration. Single CTA: Sign in with Google. After first login, brief onboarding shows the 4 agents and key features.

Acceptance criteria

- ✓ Landing page loads without authentication
- ✓ 'Sign in with Google' button triggers OAuth flow
- ✓ After first login, onboarding modal shown (3 slides: Panic Mode, Calendar, Nudges)
- ✓ Onboarding dismissable and skippable
- ✓ Onboarding not shown on subsequent logins (Firestore onboarded flag)

Tech notes

- • Public route: /
- • Protected routes: /dashboard, /panic, /task/:id, /settings
- • React Router v6 with auth guard

F12 Settings & Preferences

LOW

User settings page for notification preferences. Users can enable/disable email nudges, customize nudge thresholds, and disconnect calendar access. Account info shown.

Acceptance criteria

- ✓ Toggle email nudges on/off
- ✓ Toggle in-app notifications on/off
- ✓ Choose nudge thresholds (24h, 6h, 1h — each independently toggleable)
- ✓ View connected Google account info
- ✓ Logout button
- ✓ View ZeroHour version info

Tech notes

- • Settings stored in Firestore users collection as preferences sub-object
- • NudgeAgent reads preferences before sending nudge

6 Key User Flows

Panic Mode Flow (primary)

1. User lands on any screen, clicks red Panic button
2. Panic Mode chat interface opens
3. User types: "I have an exam in 3 hours"
4. ZeroHour (Gemini) asks follow-up: 'What subject and topics?'
5. User answers — can also say 'wait, add more info' at any point
6. After 2-3 turns, Gemini signals ready: true
7. PlannerAgent + PrioritizerAgent run (agent panel visible)
8. Plan preview shown as editable subtask cards
9. User edits/reorders if needed

10. User clicks 'Looks good — push to Calendar'
11. SchedulerAgent creates Google Calendar events
12. NudgeAgent schedules reminders
13. Completion screen with countdown + Calendar link shown

Standard Task Creation Flow

1. User clicks '+ New Task' on Dashboard
2. Task form: title, description, deadline
3. User saves — agent panel appears
4. PlannerAgent generates subtasks (3s typical)
5. PrioritizerAgent ranks them
6. Subtask cards shown below task
7. User can push to Calendar or leave as manual plan

Nudge Flow (background)

1. NudgeAgent cron fires every 15 minutes
2. Scans Firestore for tasks with deadline within 24h, 6h, 1h
3. Checks nudges collection — skips if nudge already sent
4. Writes in-app notification to Firestore
5. Sends Gmail email via Gmail API
6. Frontend onSnapshot listener updates bell badge instantly

---

7 Data Model Summary

Collection: users

Field	Type	Notes
`uid`	string PK	Google OAuth sub
`email`	string	User email
`displayName`	string	Google display name
`googleAccessToken`	string	Encrypted OAuth token
`googleRefreshToken`	string	Encrypted refresh token
`googleCalendarId`	string	Primary calendar ID
`preferences`	map	Notification preferences
`onboarded`	boolean	Onboarding shown flag
`createdAt`	timestamp	

Collection: tasks

Field	Type	Notes
`id`	string PK	UUID

userId	string FK	→ users.uid
title	string	Task title
description	string	Optional
status	enum	PENDING | IN_PROGRESS | DONE | OVERDUE
priority	enum	CRITICAL | HIGH | MEDIUM | LOW
deadline	timestamp	IST
calendarEventId	string	Nullable, FK → calendar_events
createdAt	timestamp	

Collection: subtasks

Field	Type	Notes
id	string PK	UUID
taskId	string FK	→ tasks.id
panicSessionId	string FK	Nullable → panic_sessions.id
title	string	Action-oriented title
durationMinutes	int	Gemini-estimated
status	enum	PENDING | DONE
priority	enum	From PrioritizerAgent
orderIndex	int	User-reorderable

Collection: panic_sessions

Field	Type	Notes
id	string PK	UUID
userId	string FK	→ users.uid
rawInput	string	First user message
conversationJson	string	Serialized chat history
generatedPlanJson	string	Serialized subtask array
confirmed	boolean	Whether pushed to Calendar
createdAt	timestamp	

Collection: nudges

Field	Type	Notes
id	string PK	UUID
taskId	string FK	→ tasks.id

type	enum	REMINDER_24H | REMINDER_6H | REMINDER_1H
channel	enum	IN_APP | EMAIL | BOTH
scheduledAt	timestamp	
sent	boolean	Prevents duplicates

Collection: calendar_events

Field	Type	Notes
id	string PK	UUID
googleEventId	string	Google Calendar event ID
taskId	string FK	→ tasks.id
startTime	timestamp	IST
endTime	timestamp	IST
calendarId	string	Google Calendar ID

8 REST API Reference

Auth

GET	/oauth2/authorization/google	Redirect to Google OAuth
GET	/login/oauth2/code/google	OAuth callback
GET	/api/auth/me	Get current user
POST	/api/auth/logout	Logout

Tasks

GET	/api/tasks	List all tasks for user
POST	/api/tasks	Create task → triggers PlannerAgent + PrioritizerAgent
GET	/api/tasks/{id}	Get task + subtasks + events + nudges
PUT	/api/tasks/{id}	Update task title / deadline / status
DELETE	/api/tasks/{id}	Delete task + cancel calendar events
POST	/api/tasks/{id}/confirm	Push to Calendar via SchedulerAgent
POST	/api/tasks/{id}/reprioritize	Re-run PrioritizerAgent

Panic Mode

POST	/api/panic/start	Start session: { message: string }
POST	/api/panic/{id}/reply	User reply: { message: string }
POST	/api/panic/{id}/edit	Edit plan: { subtasks: [...] }
POST	/api/panic/{id}/confirm	Confirm → SchedulerAgent + NudgeAgent
GET	/api/panic/{id}	Get session state + plan

Notifications

GET	/api/notifications	List user notifications
PUT	/api/notifications/{id}/read	Mark notification as read
PUT	/api/notifications/read-all	Mark all as read

Agent Stream

GET	/api/agents/stream/{sessionId}	SSE stream of agent log events

9 Tech Stack & Google Technologies

Google Technologies Used (Evaluation: 15%)

Gemini 2.5 Flash — AI model for all agent prompts — PlannerAgent, PrioritizerAgent, Panic Mode Q&A;

Google AI Studio — Free API key for Gemini — all agents call this endpoint

Google OAuth 2.0 — Authentication + Calendar + Gmail scope in single consent

Google Calendar API v3 — SchedulerAgent creates real calendar events

Gmail API — NudgeAgent sends deadline reminder emails

Firebase Firestore — All data storage — users, tasks, subtasks, nudges, sessions

Google Cloud Run — Deployment — Docker container, free tier, publicly accessible URL

Full Stack

Layer	Technology	Version / Notes
Frontend	React + Vite	18.x, TypeScript optional
Backend	Spring Boot	3.x, Java 17, Maven
AI Model	Gemini 2.5 Flash	Google AI Studio REST API, free tier
Database	Firebase Firestore	Spark plan, 1GB free
Auth	Google OAuth 2.0	spring-boot-starter-oauth2-client
Calendar	Google Calendar API v3	google-api-services-calendar
Email	Gmail API	google-api-services-gmail

Deployment	Google Cloud Run	Docker, 2M req/month free
HTTP Client	OkHttp	4.12.0, for Gemini API calls
JSON	Jackson	Included in Spring Boot starter
Scheduling	Spring @Scheduled	NudgeAgent cron, every 15min
Streaming	Spring SseEmitter	Agent thinking panel

10 Evaluation Matrix Mapping

Criterion	Weight	ZeroHour approach
Problem Solving & Impact	20%	Directly solves missed deadlines with AI action plans, not passive reminder
Agentic Depth	20%	4 distinct agents with real tool use: Calendar API, Gmail API, Gemini, Fires
Innovation & Creativity	20%	Panic Mode conversational flow is unique — adjustable mid-conversation, l
Usage of Google Technologies	15%	Gemini Flash, OAuth, Calendar API, Gmail API, Cloud Run, Firebase Fires
Product Experience & Design	10%	Agent thinking UI, chat-style Panic Mode, countdown timers, calm dark des
Technical Implementation	10%	Spring Boot multi-agent, SSE streaming, OAuth token refresh, batch Firest
Completeness & Usability	5%	Full flow works end-to-end: login → task → plan → edit → confirm → calen

11 6-Day Sprint Plan

Day 1 Jun 23 — **Setup & Auth**

- Spring Boot project init (Maven, all deps)
- Firebase Admin SDK + Firestore config
- Google OAuth 2.0 flow (login, token storage in Firestore)
- React + Vite frontend scaffold
- Data models (6 Java classes matching Firestore schema)
- Goal: User can sign in with Google, tokens stored

Day 2 Jun 24

Core Agents

- GeminiService (OkHttp REST client to Gemini Flash)
- FirestoreService (CRUD for all collections)
- PlannerAgent with prompt engineering
- PrioritizerAgent with urgency scoring
- AgentOrchestrator wiring agents together
- Task CRUD REST endpoints
- Goal: POST /api/tasks returns Gemini-generated subtasks

Day 3 Jun 25

Panic Mode

- Multi-turn Panic Mode conversation loop
- PanicSession Firestore model + API endpoints
- Follow-up Q&A; with Gemini (flexible, non-rigid)
- Plan preview UI in React (editable subtask cards)
- User edit/alter flow before confirmation
- Goal: Full Panic Mode conversational flow working

Day 4 Jun 26

Calendar + Nudge

- CalendarService (Google Calendar API v3)
- SchedulerAgent — create events on confirmation
- GmailService (Gmail API email sending)
- NudgeAgent — @Scheduled cron + nudge logic
- In-app notification store (Firestore + frontend)
- Goal: Confirmed plan creates Calendar events + nudges scheduled

Day 5 Jun 27

UI + Deploy

- SSE agent thinking panel (AgentStreamController + React component)
- Full Dashboard UI with filters, sorting, countdown timers
- Notification bell component
- Dockerfile + Cloud Run deploy
- React build served from Spring Boot (static resources)
- Goal: Live URL on Cloud Run, end-to-end flow working

Day 6 Jun 28

Submit

- E2E testing of all flows
- Bug fixes from testing
- README + GitHub repo cleanup
- Google Doc: problem statement, solution, features, tech
- BlockseBlock dashboard submission
- Goal: Final submit before Jun 29 deadline